Propuesta de buenas prácticas de desarrollo para SoftMarket

Diagnóstico, forma de trabajo y ejemplo de diseño

Integrante: Santiago Restrepo Cardona

Resumen

SoftMarket necesita una forma compartida de organizar el código, colaborar y revisar los cambios. Esta propuesta define pasos sencillos para trabajar en GitHub, comprobar cada mejora y dejar instrucciones útiles. El ejemplo de clientes muestra cómo separar las reglas del negocio de las herramientas para guardar datos y enviar avisos.

1. Análisis del problema

El crecimiento rápido sin acuerdos comunes hace que cada proyecto se construya de manera distinta. Se repite código, se rompen funciones al agregar mejoras y los errores llegan a producción. El aumento del 40% en errores durante los últimos tres meses muestra que hace falta detectar los problemas antes de publicar.

Problemas observados

Área	Qué ocurre	Qué puede causar
Arquitectura y diseño	Se repite lógica y una misma parte puede validar datos, guardarlos y enviar avisos.	Un cambio debe hacerse en varios lugares y alguno puede quedar distinto.
Versiones y colaboración	No hay pasos acordados para separar el trabajo y unirlo después.	Las personas pueden sobrescribir cambios o perder una versión que funcionaba.
Diseño del código	Las reglas del negocio pueden quedar amarradas a una base de datos o a un servicio de correo.	Reemplazar una herramienta obliga a cambiar muchas partes.
Calidad	Faltan pruebas, instrucciones y revisiones cuidadosas.	Los errores se descubren tarde y corregirlos cuesta más.

La solución requiere acuerdos que todo el equipo pueda seguir: proponer cada cambio por separado, revisarlo y comprobarlo antes de integrarlo.

2. Propuesta de trabajo con Git y GitHub

Git guarda el historial del proyecto. GitHub permite compartir el trabajo y revisar una propuesta antes de unirla a la versión estable.

Ramas del proyecto

Rama	Uso	Ejemplo
main	Versión estable y revisada.	Rama principal de entrega.
feature/…	Desarrollar una mejora aparte de la versión estable.	feature/1-gestion-clientes
fix/…	Corregir un error o aclarar algo existente.	fix/2-guia-revision

Reglas para trabajar

- Actualizar main antes de empezar y crear una rama con un nombre corto que describa la tarea.
- Hacer cambios pequeños y guardarlos con mensajes que expliquen el motivo: **feat(clientes): agregar registro**, **fix(clientes): validar correo** o **docs: aclarar revisión**.
- Subir la rama y abrir una solicitud de integración (pull request) hacia main. Explicar qué cambió y cómo se comprobó.
- Pedir la revisión de otra persona. Quien hizo el cambio no lo aprueba por su cuenta. Confirmar también que las pruebas pasen.
- Después de la aprobación, unir el cambio con **Squash and merge** y cerrar la rama.

Revisión y conflictos

La revisión comprueba que el cambio resuelve la tarea, evita repeticiones, incluye pruebas cuando hacen falta y mantiene las instrucciones al día. Si dos cambios chocan, actualizar la rama con la versión reciente de main, resolver juntos las diferencias, volver a ejecutar las pruebas y pedir otra revisión. No integrar mientras queden diferencias sin resolver.

El repositorio de ejemplo incluye las tres ramas solicitadas: main, feature/1-gestion-clientes y fix/2-guia-revision.

3. Ejemplo de mejora para registrar clientes

El registro de clientes debe limpiar y revisar los datos, impedir correos repetidos, guardar al cliente y confirmar el registro. Si una sola parte del programa hace todo y depende directamente de una base de datos y de un servicio de correo, cualquier cambio en esas herramientas afecta las reglas principales.

Pasos propuestos

```
1. Limpiar nombre y correo.
2. Revisar que los datos sean válidos.
3. Consultar si ya existe ese correo.
4. Si no existe, crear y guardar el cliente.
5. Avisar que el registro terminó y devolver el cliente.
```

El archivo **src/clientes/registro.py** contiene el ejemplo completo. **RegistrarCliente** coordina el alta; un acuerdo separado define cómo consultar y guardar; otro define cómo avisar. El repositorio en memoria permite mostrar el comportamiento sin instalar una base de datos.

Cómo ayuda cada principio SOLID

Principio	Aplicación	Mejora
Responsabilidad única	El servicio coordina; el repositorio guarda; el notificador avisa.	Cada parte tiene una tarea clara.
Abierto a cambios	Se puede añadir otro modo de guardar o avisar sin cambiar las reglas del alta.	Agregar opciones requiere menos cambios sobre lo existente.
Sustitución	Un repositorio de base de datos puede reemplazar al de memoria si ofrece las mismas operaciones.	Las opciones nuevas se usan sin romper el servicio.
Interfaces pequeñas	Guardar y avisar tienen acuerdos separados.	Cada parte usa solo lo que necesita.
Invertir dependencias	El servicio usa esos acuerdos, no una marca de base de datos o correo.	Las herramientas se reemplazan y prueban por separado.

Las pruebas de **tests/test_registro.py** comprueban el registro, la normalización del correo, el rechazo de un nombre vacío y la detección de correos repetidos.

4. Prácticas para cuidar la calidad

Tema	Acuerdo propuesto
Estilo	Usar nombres claros, mantener las funciones pequeñas y aplicar el mismo formato de Python. Evitar copiar lógica: moverla a un lugar común.
Pruebas	Comprobar cada regla y los recorridos principales. Antes de integrar un cambio, ejecutar las pruebas existentes y añadir una para cada comportamiento nuevo o corregido.
Documentación	Mantener instrucciones para entender y ejecutar el ejemplo. Explicar cómo crear ramas, pedir revisiones y resolver conflictos.
Revisión	Comprobar que la tarea esté explicada, que las pruebas se hayan ejecutado y que las instrucciones estén actualizadas.
Control de calidad	Revisar los cambios, ejecutar las pruebas y confirmar que el programa inicia antes de integrar o publicar.

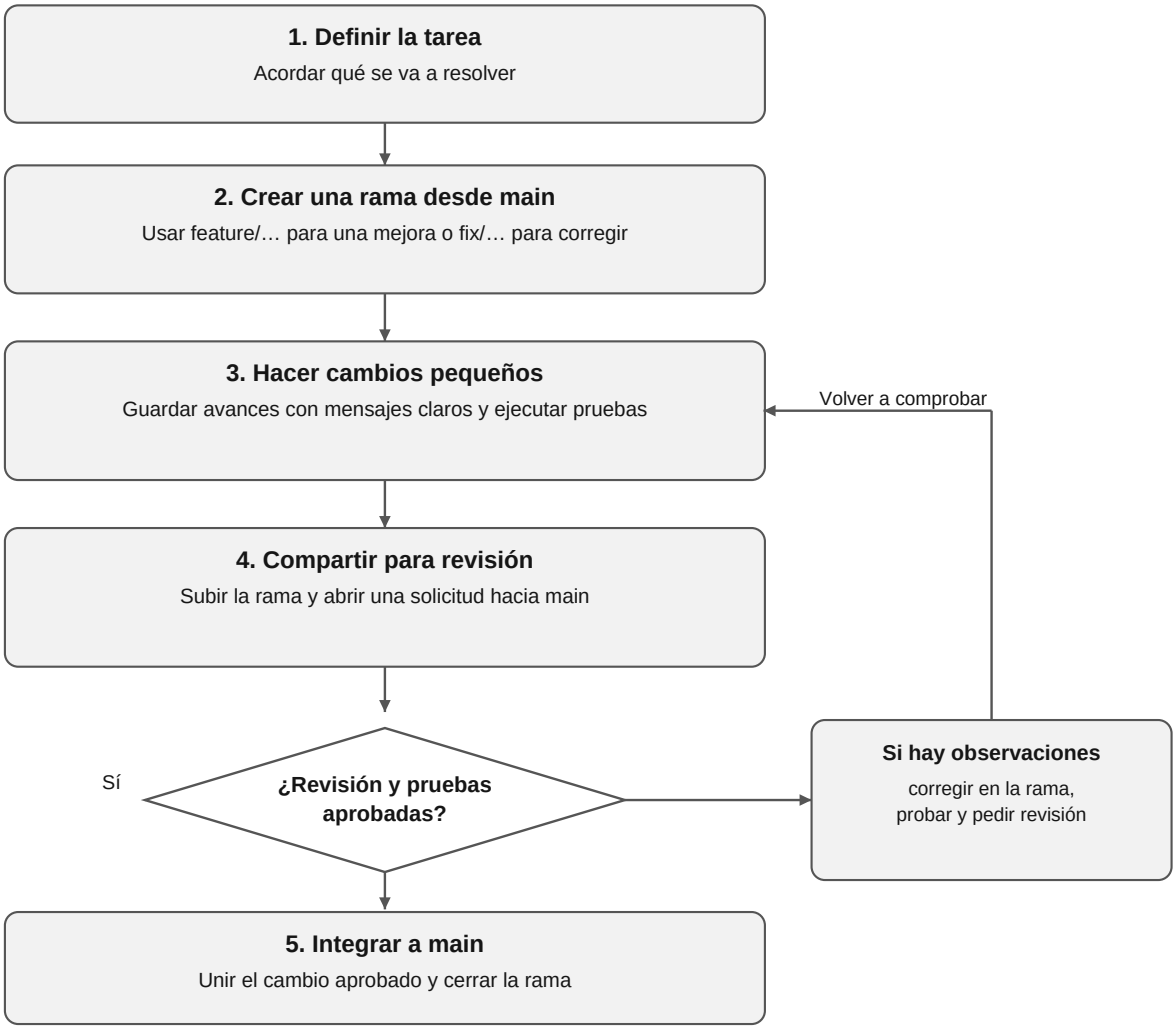
Lista antes de integrar

- ¿La solicitud explica el motivo del cambio y se relaciona con una tarea?
- ¿Otra persona revisó el cambio y quedaron respondidas sus observaciones?
- ¿Se ejecutaron las pruebas y se actualizaron las instrucciones necesarias?

Con estos pasos, el equipo puede detectar errores antes de publicar y conservar un historial claro de lo que cambió y por qué.

5. Flujo completo de versionamiento

El trabajo empieza con una tarea y una rama separada. La propuesta se revisa y se prueba antes de integrarla. Si hay observaciones, se corrige en la misma rama y se solicita una nueva revisión.


1. Definir la tarea
Acordar qué se va a resolver
2. Crear una rama desde main
Usar feature/… para una mejora o fix/… para corregir
3. Hacer cambios pequeños
Guardar avances con mensajes claros y ejecutar pruebas
Volver a comprobar
4. Compartir para revisión
Subir la rama y abrir una solicitud hacia main
Sí
¿Revisión y pruebas aprobadas?
Si hay observaciones
corregir en la rama,
probar y pedir revisión
5. Integrar a main
Unir el cambio aprobado y cerrar la rama

6. Entregables y cierre

El repositorio público acompaña este informe con el código fuente visible, sus pruebas, la guía de trabajo, la plantilla para solicitar revisión y los informes en PDF y formato editable.

Ramas de ejemplo

- **main:** versión estable del proyecto.
- **feature/1-gestion-clientes:** mejora para registrar clientes.
- **fix/2-guia-revision:** corrección a la guía de revisión.

Archivos principales

- **src/clientes/registro.py:** ejemplo de registro con tareas separadas.
- **tests/test_registro.py:** pruebas del comportamiento del registro.
- **docs/flujo-git.md:** pasos del trabajo con ramas y revisión.
- **.github/PULL_REQUEST_TEMPLATE.md:** guía para describir y revisar cambios.

Repositorio

github.com/santiago1515m-del/softmarket-buenas-practicas

Conclusión

Separar el trabajo en ramas, pedir una revisión independiente y ejecutar pruebas ayuda a detectar errores antes de que lleguen a producción. El ejemplo de clientes muestra cómo mantener las reglas del negocio separadas de las herramientas para guardar datos y enviar avisos. Si el equipo sigue estos acuerdos en todos sus proyectos, podrá corregir y ampliar el sistema con menos riesgo.